CHAPTER 3

Number Representation & Arithmetic

Unsigned · sign-magnitude · 1's and 2's complement · overflow · IEEE 754 single and double · special values

QUICK TRICK

n-bit **2's complement** range: $\mathbf{-2^{n-1} \ldots 2^{n-1} - 1}$. Negate = **invert + 1**. Overflow = carry into MSB ≠ carry out of MSB. IEEE single: **1 | 8 | 23**, bias **127**; double: **1 | 11 | 52**, bias **1023**. Value = $(-1)^s \times 1.f \times 2^{e - bias}$.

(This topic is listed under Digital Logic in the official syllabus; it is placed here because COA uses it in ALU and data-path questions.)

3.1 Integer representations

● **LEVEL 1 · BEGINNER**

Scheme	n-bit range	Zeros	Negate
Unsigned	$0 \ldots 2^n - 1$	1	–
Sign-magnitude	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	2 (+0, −0)	flip the sign bit
1's complement	$-(2^{n-1} - 1) \ldots +(2^{n-1} - 1)$	2	invert all bits
2's complement	$\mathbf{-2^{n-1} \ldots 2^{n-1} - 1}$	1	invert, then add 1

- **2's complement value** = $-b_{n-1} \times 2^{n-1}$ + (the other bits as usual). 8-bit 0x9C = 10011100 = −128 + 16 + 8 + 4 = **−100**.
- **Sign extension:** to widen a 2's complement number, copy the sign bit into the new high bits.
- **Excess-K (biased):** stored = value + K. Used for floating-point exponents.
- The 2's complement range has **one more negative** number than positive (the most negative has no positive partner).

Example. −25 in 8 bits: 25 = 00011001 → invert 11100110 → +1 = **11100111**.

3.2 Addition, subtraction, overflow

● **LEVEL 2 · EXAM**

- Subtract A − B as **A + (−B)** = A + B' + 1. The same adder does both.
- **Overflow** (signed): adding two positives gives a negative, or two negatives give a positive. Test: **V = C_{in} to MSB XOR C_{out} of MSB**.
- **Carry out** (C flag) signals overflow for **unsigned** numbers. Adding numbers of opposite sign never overflows.

Example. 8-bit 0x7F + 0x01 = 0x80: two positives gave a negative, so **overflow**. 4-bit 0110 + 0101 = 1011: carry into MSB = 1, carry out = 0, so V = 1.

3.3 IEEE 754 floating point

● **LEVEL 2 · EXAM**

```
Single (32 bit):  | s (1) | exponent e (8) | fraction f (23) |   bias 127
Double (64 bit):  | s (1) | exponent e (11)| fraction f (52) |   bias 1023
Normal number  : value = (-1)^s x 1.f x 2^(e - bias)       (hidden leading 1)
```

Exponent e	Fraction f	Meaning (single)
0	0	±0
0	≠ 0	Denormal: $(-1)^s \times 0.f \times 2^{-126}$
1 ... 254	any	Normal number
255	0	±Infinity

Exponent e	Fraction f	Meaning (single)
255	≠ 0	NaN

Encode 5.75. $5.75 = 101.11_2 = 1.0111 \times 2^2$. e = 2 + 127 = 129 = 10000001. Bits: 0 | 10000001 | 0111000...0 → **0x40B80000**.

Encode −0.15625. $0.15625 = 0.00101_2 = 1.01 \times 2^{-3}$. e = 124 = 01111100. Bits: 1 | 01111100 | 0100...0 → **0xBE200000**.

Decode 0xC1200000. 1 | 10000010 | 0100...0: sign −, e = 130 → exponent 3, mantissa 1.01 = 1.25. Value = −1.25 × 8 = **−10.0**.

Quantity (single)	Value
Largest finite	$(2 - 2^{-23}) \times 2^{127} \approx 3.4 \times 10^{38}$
Smallest positive normal	2^{-126}
Smallest positive denormal	2^{-149}
Precision	24 bits (23 + hidden 1) ≈ 7 decimal digits
NaN patterns	$2 \times (2^{23} - 1) = 2^{24} - 2$

3.4 Floating-point addition (steps)

● LEVEL 3 · GATE

- **Align** exponents: shift the mantissa of the smaller number right by the difference.
- **Add** (or subtract) the mantissas.
- **Normalise** the result to 1.f form (adjust exponent).
- **Round**, and check overflow/underflow.

QUICK TRICK

Numbers such as 0.1 cannot be stored exactly in binary, so floating-point results carry small rounding errors. Between 2^k and 2^{k+1} there are exactly $\mathbf{2^{23}}$ single-precision values.

3.5 Common mistakes

COMMON MISTAKE

- Forgetting the hidden 1 (the mantissa is 1.f, not 0.f, for normal numbers).
- Using bias 128 instead of 127 (or 1024 instead of 1023).
- Treating the carry flag as signed overflow.
- Taking the 2's complement range as symmetric.

GATE QUICK CHECK — Chapter 3

```
2s compl: -2^(n-1)..2^(n-1)-1     negate = invert+1     V = C_into_MSB XOR C_out_MSB
single 1|8|23 bias127   double 1|11|52 bias1023     value = (-1)^s * 1.f * 2^(e-bias)
e=0,f=0 zero | e=0,f!=0 denormal | e=max,f=0 inf | e=max,f!=0 NaN
5.75 = 0x40B80000     -10.0 = 0xC1200000     0.75 = 0x3F400000
```

Practice questions

● LEVEL 1 · BEGINNER

Q1. Range of an 8-bit 2's complement number.

Q2. Write −25 as an 8-bit 2's complement number.

Q3. Decimal value of the 8-bit 2's complement number 0x9C.

Q4. Exponent bias for IEEE single and double precision.

● LEVEL 2 · EXAM

Q5. 8-bit 2's complement: 0x7F + 0x01. Result and flags?

Q6. Decode the single-precision number 0xC1200000.

Q7. Encode 0.75 in IEEE single precision (hex).

Q8. How many distinct 32-bit patterns represent NaN?

● **LEVEL 3 · GATE**

Q9. Smallest positive normal and denormal single-precision numbers.

Q10. In 4-bit 2's complement compute 0110 + 0101. Carry into and out of the MSB, and the overflow flag.

Q11. How many 8-bit patterns exist in 1's complement for zero, and what is the range?

Q12. How many single-precision numbers lie in the interval [1, 2)?

Answers

Q	Solution
1	**−128 … +127**.
2	**11100111**.
3	156 − 256 = **−100**.
4	**127** and **1023**.
5	Result 0x80 (−128). Signed **overflow** (two positives gave a negative); no carry out.
6	**−10.0** (see 3.3).
7	0.75 = 1.1 × 2^{-1}; e = 126 = 01111110. Bits 0 | 01111110 | 1000…0 = **0x3F400000**.
8	e = 255, f ≠ 0: 2 × (2^{23} − 1) = **16,777,214**.
9	**2^{-126}** and **2^{-149}**.
10	Sum 1011. Carry into MSB = 1, carry out = 0, so **V = 1** (overflow: 6 + 5 = 11 > 7).
11	Two zeros (00000000, 11111111). Range **−127 … +127**.
12	**2^{23} = 8,388,608** (exponent fixed, 23 fraction bits free).

Next: Chapter 4 — Instruction Set and Instruction Formats.

AMJESH singhamjesh.github.io